arXiv:2504.14349v2 [quant-ph] 24 Apr 2025

Application of an upsampling algorithm to quantum state preparation of continuous and discrete probability distributions

R. P. Erickson[1,*]

[1] *Wells Fargo*

(Dated: April 28, 2025)

Upsampling of time series data is often associated with classical fast Fourier transform analysis, but it also can be employed in the preparation of a quantum state vector. The quantum circuit of preparation derived from functional data sampled in this way exhibits exponential gate depth, like other divide-and-conquer algorithms, but is transformable to an equivalent circuit of polylogarithmic gate depth. In this study, we derive a polylogarthmic upsampling algorithm for construction of a state vector with amplitudes that are square roots of probabilities sampled from a continuous probability distribution having support over the entire real line. As examples, we prepare state vectors associated with Gaussian and Laplace distributions; state vectors for other continuous probability distributions such as Cauchy and Student's t follow in a similar manner. We also extend the algorithm to include preparation of a state vector whose amplitudes are square roots of an arbitrary distribution of discrete probabilities. Our analyses focus on univariate distributions, but can be extended to multivariate forms. The polylogarithmic upsampling algorithm has financial and scientific application.

I. INTRODUCTION

Many financial, scientific, and machine-learning quantum computing applications require loading of classical information into a state vector. Repeated use of this entangled quantum state in a calculation necessitates reloading due to the no-cloning restriction imposed by the nature of quantum mechanics [1]. Additionally, limitations exist on both dimension of state vector and time to completion of the underlying calculation due to quantum decoherence, external noise and stray fields, and scaling issues that inhibit control and precision, such as the overhead of large numbers of physical qubits needed to implement quantum error correction schemes. These restrictions impose a high level of efficiency on any quantum algorithm used to prepare the state vector [2]. To address these challenges, recent efforts have included the use of numerical integration [3], quantum generative adversarial networks [4], black-box (oracle) approaches [5], matrix product states [6], and other methods emphasizing fault tolerance [7].

In this study we apply an upsampling method of data capture, most often associated with classical fast Fourier transform analysis of discrete samples of time series, to the construction of a quantum algorithm that can be used to prepare a state vector whose amplitudes are obtained from the discretization of a continuous function. The algorithm derived is a divide-and-conquer algorithm [8] composed of controlled y-rotation gates [9], with exponential gate depth, but it can be transformed to an equivalent circuit of polylogarthmic gate depth [10] using a method of quantum forking [11]. In our analysis we consider the specific univariate examples of Gauss and Laplace; application to other continuous probability distributions, such as Cauchy and Student's t, follows from the formulas presented. We also extend our study to include the case of distributions of discrete probabilities.

In our method of quantum state preparation, we adapt elements of the approach of Kitaev and Webb [12], in particular, defining a periodic equivalent of a continuous probability density function (PDF). The quantum state vector is prepared such that the probability of measurement of a given multi-qubit state is proportional to the value of a point on the periodic function. We capture the functional dependence of the PDF within the computational resource using conventional time-series-like upsampling, defining a sampling interval (and a limiting Nyquist frequency) as a function of the employed number of qubits.

Specifically, if a univariate PDF is supported on the entire real line then it can be denoted by continuous function $P(x)$, where $x \in \mathbb{R}$ and the probability of occurrence of x on the interval of x to $x + dx$ is $P(x)\,dx$. For a computational resource consisting of n qubits, we define unsigned integers $0 \le i_n < 2^n$ and represent them in terms of a set of bit values $i_m^{(n)} \in \{0, 1\}$, where $0 \le m < n$, such that

$$i_n = \sum_{m=0}^{n-1} 2^m \, i_m^{(n)} \; ; \quad i_0 \equiv 0. \tag{1}$$

In the little endian convention, used throughout this study, an n-qubit state can be defined as $|i_n\rangle = |i_{n-1}^{(n)}, \ldots, i_0^{(n)}\rangle$, where $i_{n-1}^{(n)}$ is the most significant bit.

Unlike [12], we capture $P(x)$ within a sampling window, w, and map only integers i_n (supporting the n-qubit system) to real values x_n via the linear formula

$$x_n = x_o + \Delta x_n \cdot i_n \; ; \tag{2}$$

$$\Delta x_n = \frac{w}{2^n}, \quad x_o = \bar{x} + w\,(\zeta - 1)\,/2, \tag{3}$$

* Electronic address: Robert.Erickson2@wellsfargo.com

where $P(\bar{x}) = \max P(x)$, Δx_n is the sampling interval, and $f_n = 1/(2\Delta x_n)$ is the Nyquist frequency.

Specific to our approach, the parameter ζ, where $0 \leq \zeta < 1/2^{n-1}$, can be chosen at random and introduces a small shift in a sampling window that would otherwise be centered at $\bar{x}$. Two state vectors of equal dimension prepared with different values of ζ will consist of distinctly different sets of amplitudes/probabilities, preventing aliasing when resampling the PDF [13]. Figure 1(a) illustrates our upsampling approach as a function of increasing n, for fixed values of w and ζ.

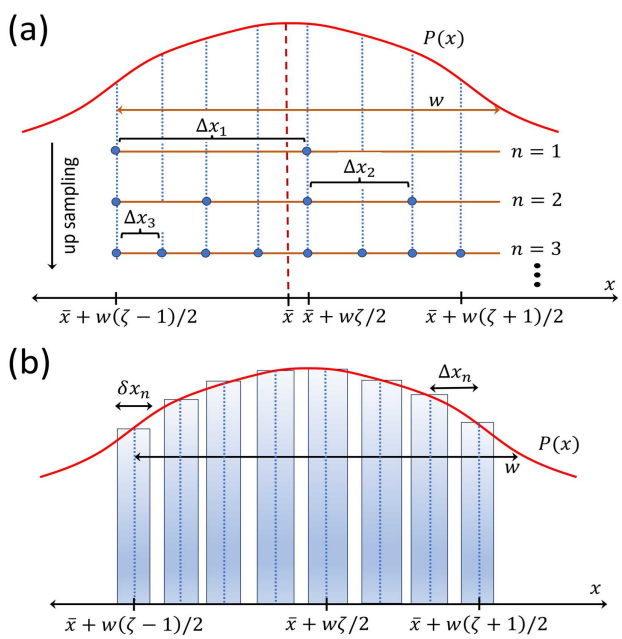

FIG. 1. (a) Upsampling capture of PDF, $P(x)$ (solid red curve), as a function of increasing number of qubits, n, for fixed values of window width, w, and parameter ζ. For each n, the sampling window is centered about $x = \bar{x} + w\zeta/2$, where $P(\bar{x}) = \max P(x)$ and contains 2^n points (blue dots) uniformly separated in distance by Δx_n, the sampling interval. (b) Plot of $P(x)$ vs x showing how sample probabilities, $|\langle i_n|\psi_n\rangle|^2$ of (11), corresponding to a system of n qubits, are represented by areas under curve $P(x)$. Note that normalization factor, δx_n, and sampling interval, Δx_n, are different quantities.

We write the state vector to be prepared, $|\psi_n\rangle$, as a superposition of qubit states, $|i_n\rangle$, viz.

$$|\psi_n\rangle = \sum_{i_n} \xi_n(i_n)\, |i_n\rangle\,, \tag{4}$$

where $\xi_n(i_n)$ is an amplitude, and the sum is over all unsigned integers $0 \leq i_n < 2^n$. Similar to [12], invoking the map of (2) so that $\xi_n(i_n) = \xi_n(x_n(i_n)) = \xi_n(x_n)$, we define, for any $x \in \mathbb{R}$, that

$$\xi_n(x) = \sqrt{\delta x_n \sum_{j=-\infty}^{\infty} P(x + jw)}, \tag{5}$$

such that $\xi_n(x)$ is periodic in w, viz.

$$\xi_n(x + w) = \xi_n(x). \tag{6}$$

Then, requiring $\langle\psi_n|\psi_n\rangle = 1$, the normalization factor, δx_n, satisfies the constraint

$$\sum_{j=-\infty}^{\infty} \sum_{x_n} P(x_n + jw)\, \delta x_n = 1. \tag{7}$$

Several observations can be made about the above construction of $|\psi_n\rangle$. As stated in (6), the first observation is that $\xi_n(x_n)$ of (5) is periodic in x_n with period w. A second observation is the limiting behavior of (7), viz.

$$\lim_{n\to\infty} \sum_{j=-\infty}^{\infty} \sum_{x_n} P(x_n + jw)\, \delta x_n$$
$$= \sum_{j=-\infty}^{\infty} \int_{-w/2}^{w/2} P(x + jw)\, dx = \int_{-\infty}^{\infty} P(x)\, dx = 1, \tag{8}$$

where δx_n becomes infinitesimally small as $n \to \infty$, such that the sum over x_n can be replaced by an integral. A third observation, due to the support of $P(x)$ over all $x \in \mathbb{R}$, is that when w is chosen sufficiently large then

$$\sum_{j=-\infty}^{\infty} P(x_n + jw) \cong P(x_n), \tag{9}$$

to very good approximation, such that

$$\xi_n(x_n) \cong \sqrt{P(x_n)\, \delta x_n}, \tag{10}$$

$$|\langle i_n|\psi_n\rangle|^2 \cong P(x_n)\, \delta x_n. \tag{11}$$

A fourth observation is that the quantities δx_n and Δx_n are not the same; the former is a normalization factor while the latter is the sampling interval. Figure 1(b) provides a graphical illustration of these last two observations.

In what follows we present the upsampling algorithm for preparing $|\psi_n\rangle$ of (4), applicable to the Gaussian and Laplace probability distributions, specifically. We then extend our approach to show how a state vector corresponding to a set of discrete probabilities can be prepared by this same algorithm. Next, we apply the concept of quantum forking [11] to our algorithm to transform it to a polylogarithmic form, following on with several example calculations. We conclude with several final remarks.

II. THEORY

We first derive the upsampling quantum algorithm applicable to continuous probability distributions. We then use this result to show how state vectors associated with discrete probabilities can be prepared. Lastly, we provide a brief overview of the quantum forking algorithm as it applies to upsampling.

A. The Upsampling Quantum Circuit

In the first step of the derivation, we write the normalization factor, δx_n, in its simplest form using the summation identity

$$\sum_{j=-\infty}^{\infty} P(x+jy) = \sum_{j=-\infty}^{\infty} P\left(x+2jy\right) + \sum_{j=-\infty}^{\infty} P\left(x+(2j+1)\,y\right) \;; \quad x,y\in\mathbb{R}. \tag{12}$$

Solving for δx_n in (7) we initially have

$$\delta x_n = \frac{1}{\sum_{j=-\infty}^{\infty}\sum_{i_n} P(x_n(i_n)+jw)}, \tag{13}$$

but the sum over i_n can be broken up into pairs of terms involving $P(x_n+jw)$ and $P(x_n+\frac{1}{2}w+jw)$. In other words, dividing the sum into two parts with respect to the most significant bit, we can write

$$\delta x_n = \left\{ \sum_{j=-\infty}^{\infty}\sum_{i_{n-1}} \Big[P(x_n(i_{n-1})+jw) + P(x_n(i_{n-1})+(2j+1)\,w/2) \Big] \right\}^{-1}, \tag{14}$$

then invoke (12) to obtain the reduction

$$\delta x_n = \frac{1}{\sum_{j=-\infty}^{\infty}\sum_{i_{n-1}} P(x_n(i_{n-1})+jw/2)}. \tag{15}$$

We repeat this process, breaking up the sum over i_{n-1} into terms involving $P(x_n+jw/2)$ and $P(x_n+\frac{1}{4}w+jw/2)$, and again applying (12). Performing this action $n-1$ times, we arrive at the final expression

$$\delta x_n = \frac{1}{\sum_{j=-\infty}^{\infty} P(x_o+jw/2^n)}. \tag{16}$$

Similarly, in a second step, we can reduce the expression of $|\psi_n\rangle$ of (4) to a form amenable to quantum computation. Specifically, making use of (5), initially we have

$$|\psi_n\rangle = \sqrt{\delta x_n}\sum_{i_n}\sqrt{\sum_{j=-\infty}^{\infty} P(x_n(i_n)+jw)}\cdot|i_n\rangle\,. \tag{17}$$

For the specific case of $n=1$, we can leverage (16) and write

$$|\psi_1\rangle = \sqrt{\frac{\sum_{j=-\infty}^{\infty} P(x_0+jw)}{\sum_{j=-\infty}^{\infty} P(x_0+jw/2)}}\cdot|0\rangle + \sqrt{\frac{\sum_{j=-\infty}^{\infty} P(x_0+(2j+1)\,w/2)}{\sum_{j=-\infty}^{\infty} P(x_0+jw/2)}}\cdot|1\rangle\,. \tag{18}$$

The summation identity of (12) allows us to introduce angles of rotation, $\theta_{n,m}(i)$, via the definitions

$$\cos\frac{\theta_{n,m}(i)}{2} = \sqrt{\frac{\sum_{j=-\infty}^{\infty} P(x_n(i)+jw/2^{m-1})}{\sum_{j=-\infty}^{\infty} P(x_n(i)+jw/2^{m})}}, \tag{19}$$

$$\sin\frac{\theta_{n,m}(i)}{2} = \sqrt{\frac{\sum_{j=-\infty}^{\infty} P(x_n(i)+(2j+1)\,w/2^{m})}{\sum_{j=-\infty}^{\infty} P(x_n(i)+jw/2^{m})}}, \tag{20}$$

such that for $n=1$ and $m=1$ we have

$$|\psi_1\rangle = \cos\frac{\theta_{1,1}(0)}{2}\cdot|0\rangle + \sin\frac{\theta_{1,1}(0)}{2}\cdot|1\rangle\,. \tag{21}$$

More generally, for the case $n>1$, an initial divide of the sum over $0\le i_n<2^n$ into two parts, based on the value of the most significant bit, allows us to write (17) as

$$|\psi_n\rangle = \sqrt{\delta x_n}\sum_{i_{n-1}}\left(\sqrt{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-1})+jw)}\cdot|0\rangle + \sqrt{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-1})+(2j+1)\,w/2)}\cdot|1\rangle\right)\otimes|i_{n-1}\rangle\,. \tag{22}$$

Then, making use of (12), with the aid of (19) and (20), equation (22) can be expressed as

$$|\psi_n\rangle = \sqrt{\delta x_n}\sum_{i_{n-1}}\left(\cos\frac{\theta_{n,1}(i_{n-1})}{2}\cdot|0\rangle + \sin\frac{\theta_{n,1}(i_{n-1})}{2}\cdot|1\rangle\right) \otimes\sqrt{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-1})+jw/2)}\cdot|i_{n-1}\rangle\,. \tag{23}$$

Again, dividing the sum into two parts with respect to the most significant bit, and making use of the three equations (12), (19) and (20), yields

$$|\psi_n\rangle = \sqrt{\delta x_n}\sum_{i_{n-2}}\Bigg[\left(\cos\frac{\theta_{n,1}(i_{n-2})}{2}\cdot|0\rangle + \sin\frac{\theta_{n,1}(i_{n-2})}{2}\cdot|1\rangle\right)\otimes\cos\frac{\theta_{n,2}(i_{n-2})}{2}\cdot|0\rangle + \left(\cos\frac{\theta_{n,1}(i_{n-2}+2^{n-2})}{2}\cdot|0\rangle + \sin\frac{\theta_{n,1}(i_{n-2}+2^{n-2})}{2}\cdot|1\rangle\right) \otimes\sin\frac{\theta_{n,2}(i_{n-2})}{2}\cdot|1\rangle\Bigg] \otimes\sqrt{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-2})+jw/4)}\cdot|i_{n-2}\rangle\,. \tag{24}$$

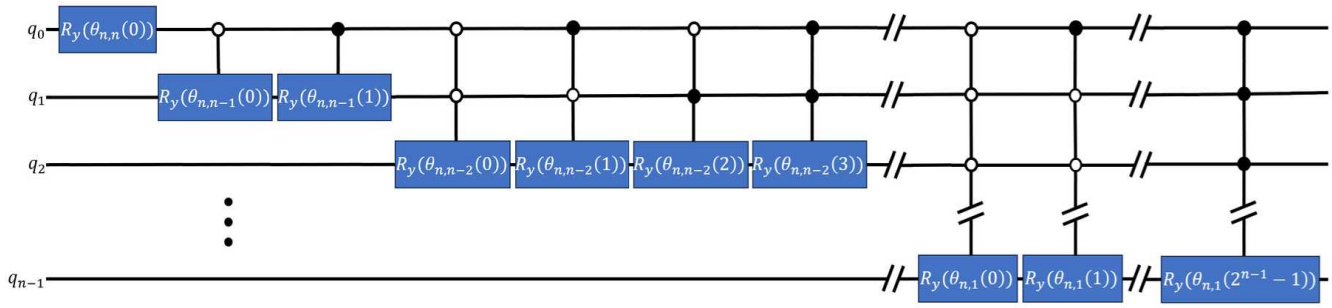

FIG. 2. Circuit diagram for the n-qubit upsampling algorithm, parameterized by rotation angles $\theta_{n,m}(i)$, as defined in (26). The circuit consists of sequences of controlled y-rotation gates. Specifically, qubit q_m, where $0 \leq m < n$, includes 2^m such gates where the single-qubit y rotations are of the form $R_y(\theta_{n,n-m}(i))$, where $i = 0, 1, \ldots, 2^m - 1$. The open (closed) vertices represent negative (positive) controls, as determined from the base-2 representation of integer-argument i of a given rotation.

Repeating these steps, for a total of $n-1$ iterations, and applying (16) in the last iteration, yields a result that can be expressed as a sum of Kronecker products, viz.

$$|\psi_n\rangle = \sum_{i_n} \bigotimes_{m=1}^{n} \cos\left(-\frac{\pi}{2} i_{n-m}^{(n)} + \frac{1}{2}\theta_{n,m}(i_{n-m})\right) |i_{n-m}^{(n)}\rangle\,, \tag{25}$$

where we recall the notation defined in (1). Also, the definition in (19) implies

$$\theta_{n,m}(i_{n-1}) = 2 \times \arccos\sqrt{\frac{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-1}) + jw/2^{m-1})}{\sum_{j=-\infty}^{\infty} P(x_n(i_{n-1}) + jw/2^{m})}}, \tag{26}$$

where we have replaced i by i_{n-1} since $0 \leq i < 2^{n-1}$. The n-qubit circuit diagram corresponding to (25) is depicted in Fig. 2 in terms of y-rotation gates parameterized by rotation angles $\theta_{n,m}(i)$, where $m = 1, 2, \ldots, n$. The algorithm represented by the circuit of Fig. 2 is a form of divide-and-conquer algorithm [9].

B. The Application to Discrete Probabilities

The quantum circuit of Fig. 2 can be leveraged for preparation of state vectors whose amplitude coefficients are the square roots of discrete probabilities. Specifically, consider a discrete distribution of probabilities of the form

$$\mathscr{P}(i_n) = \frac{f(i_n)}{\sum_{i_n} f(i_n)}\ ; \quad \sum_{i_n} \mathscr{P}(i_n) = 1, \tag{27}$$

where $f(i_n)$ is a real number dependent on integer i_n, with corresponding quantum state vector

$$|\phi_n\rangle = \sum_{i_n} \sqrt{\mathscr{P}(i_n)}\ |i_n\rangle\ ; \quad \langle\phi_n|\phi_n\rangle = 1. \tag{28}$$

It is not possible to prepare (28) from the set of unitary gate operations depicted in Fig. 2, except when $n = 1$. In this special case, because $\mathscr{P}(1)$ is the compliment of $\mathscr{P}(0)$, we have

$$|\phi_1\rangle = \sqrt{\mathscr{P}(0)}\ |0\rangle + \sqrt{1-\mathscr{P}(0)}\ |1\rangle\,, \tag{29}$$

such that we can define

$$\theta_{1,1}(0) = 2\arccos\sqrt{\mathscr{P}(0)}, \tag{30}$$

$$|\psi_1\rangle = \cos\frac{\theta_{1,1}(0)}{2}\cdot|0\rangle + \sin\frac{\theta_{1,1}(0)}{2}\cdot|1\rangle = |\phi_1\rangle\,. \tag{31}$$

The form of (29) suggests introducing an auxiliary state vector defined from complimentary probabilities, viz.

$$|\phi_n^*\rangle = \sum_{i_n} \sqrt{1-\mathscr{P}(i_n)}\ |i_n\rangle\ ; \quad \langle\phi_n^*|\phi_n^*\rangle = 2^n - 1, \tag{32}$$

then defining the actual state vector of preparation as

$$|\psi_n\rangle = \frac{1}{\sqrt{2^{n-1}}}\left(|0\rangle\otimes|\phi_{n-1}\rangle + |1\rangle\otimes|\phi_{n-1}^*\rangle\right)\ ; \quad n > 1. \tag{33}$$

Now we have a state vector in (33) that can be prepared via the quantum circuit of Fig. 2. To see this, we first note that it has a corresponding PDF given by

$$P(x) = \frac{1}{2^{n-1}}\sum_{i_{n-1}}\Big[\mathscr{P}(i_{n-1})\,\delta(x - x_n(i_{n-1})) + (1-\mathscr{P}(i_{n-1}))\,\delta(x - x_n(2^{n-1}+i_{n-1}))\Big]\ ; \quad n > 1, \tag{34}$$

as shown in Appendix A. Here, we have reused the definition of (2) for $x_n(i)$, but with $\bar{x} = 0$, $\zeta = 0$, and w of arbitrary length. Also, we introduced the Dirac delta function, $\delta(x)$, noting that (i) $P(x) = 0$ when $|x| > w/2$ by construction, such that (9) is satisfied exactly rather than approximately, i.e.,

$$P(x_n + jw) = P(x_n)\,\delta_{j,0}, \tag{35}$$

and (ii) $P(x)$ is normalized to unity, when integration is over all $x \in \mathbb{R}$.

Parameters $\theta_{n,m}(i)$ are computed from (26) using (34). In this case, as in Appendix A, the Dirac delta functions of (34) are replaced with Gaussians, i.e.,

$$\delta(x) = \lim_{\epsilon \to 0} \frac{1}{\sqrt{2\pi\epsilon^2}} e^{-x^2/2\epsilon^2}. \tag{36}$$

As shown in Appendix B, we evaluate sums over j as $\epsilon \to 0$ in (26) and obtain

$$\theta_{n,m}(i_{n-1}) = \begin{cases} 2\arccos\sqrt{\mathscr{P}(i_{n-1})} & ; \ m = 1 \\ \pi/2 & ; \ 1 < m \leq n \end{cases}. \tag{37}$$

If (37) is substituted into (25) then we arrive at an expression specific to discrete probability distributions given by

$$|\psi_n\rangle = \frac{1}{\sqrt{2^{n-1}}} \sum_{i_n} \cos\left(-\frac{\pi}{2} i_{n-1}^{(n)} + \frac{\theta(i_{n-1})}{2}\right) |i_n\rangle, \tag{38}$$

where we have simplified notation to include only the angles of the y-rotation gates corresponding to $m = 1$, viz.

$$\theta(i_{n-1}) = 2\arccos\sqrt{\mathscr{P}(i_{n-1})}. \tag{39}$$

Figure 3 shows the circuit diagram for preparing the state vector of (38); gates corresponding to $\pi/2$ rotation angle reduce to Hadamard gates, H, and even multiples of these gates reduce further to identity operations. It is easy to see that (33) and (38) are both expressions of the same state vector, $|\psi_n\rangle$. For example, comparing the two expressions, we have

$$|\phi_n\rangle = \sum_{i_n} \cos\left(\frac{\theta(i_n)}{2}\right) |i_n\rangle = \sum_{i_n} \sqrt{\mathscr{P}(i_n)}\, |i_n\rangle. \tag{40}$$

C. The Quantum Forking Algorithm

We have shown how to construct a divide-and-conquer upsampling algorithm for quantum state preparation. The quantum circuit corresponding to this algorithm, as depicted in Fig. 2, exhibits exponential gate depth, but can be transformed to an equivalent circuit of polylogarthmic gate depth, as described by Araujo and coworkers [10], using a method of quantum forking [11]. The source code of Araujo and coworkers can be found on the GitHub site [14].

Compared to the n-qubit divide-and-conquer upsampling algorithm, the method of [10] introduces a control qubit, $d = 2^n - 1$ ancilla qubits, and a set of $d - 1$ controlled swap gates, as depicted in Fig. 1 of [11], reducing time complexity by a factor of d, with a gate depth of $O(\log_2^2(d))$. In the examples that follow, we convert the upsampling algorithm to the quantum-forking algorithm, explicitly showing the quantum-forking circuit used in our calculations.

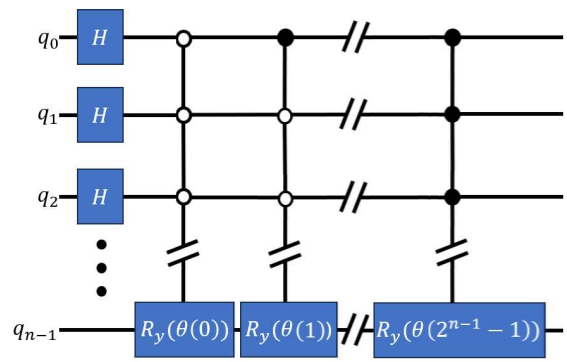

FIG. 3. The n-qubit circuit diagram of state vector $|\psi_n\rangle$, of (33), parameterized by rotation angles $\theta(i)$, as defined in (39). Hadamard gates, H, are applied to the lesser significant qubits, $q_0, q_1, \ldots, q_{n-2}$, while controlled y-rotations, by angle $\theta(i)$, are applied thereafter, in sequence, with $i = 0, 1, \ldots, 2^{n-1} - 1$. The open (closed) vertices represent negative (positive) controls, as determined from the base-2 representation of integer-argument i of a given rotation.

III. EXAMPLES

We present simulations of state-vector preparation, for both continuous and discrete probability examples. Continuous-probability examples include state vectors of Gaussian and Laplace distributions while a discrete-probability example shown is that of the binomial distribution. We include illustrations of the specific quantum-forking quantum circuits used in our calculations.

A. Continuous Probability Distributions

1. *The Gaussian Distribution*

The univariate normal, or Gaussian, probability density can be expressed as

$$P(x|\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right], \tag{41}$$

where the upsampling center of capture on the line of support is $\bar{x} = \mu$, with

$$\mu = \int_{-\infty}^{\infty} x P(x|\mu,\sigma)\, dx, \tag{42}$$

$$\sigma^2 = \int_{-\infty}^{\infty} (x-\mu)^2 P(x|\mu,\sigma)\, dx. \tag{43}$$

Figure 4(a) shows the sampling measurement of a state vector prepared using the quantum-forking circuit of Fig. 4(b). The quantum-forking circuit was the result of converting an upsampling circuit of $n = 3$ qubits.

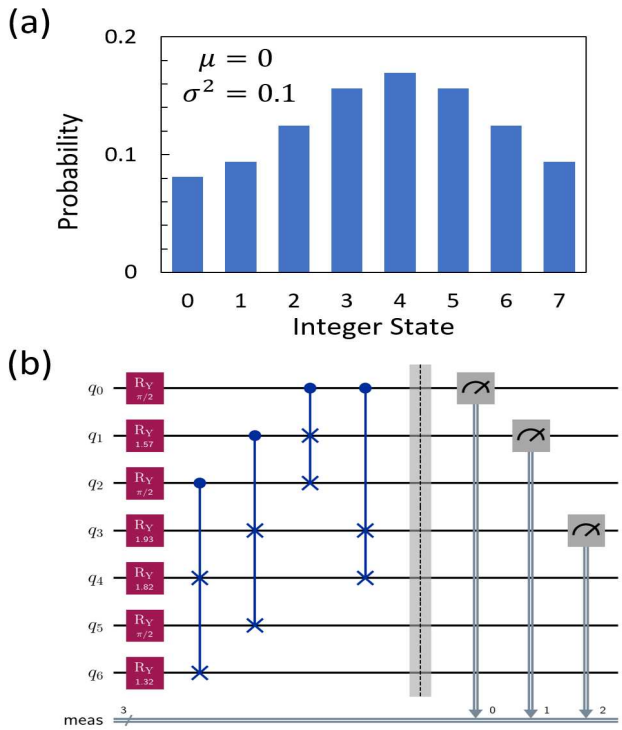

FIG. 4. (a) Gaussian distributed probabilities corresponding to a state vector prepared via the quantum-forking circuit of panel (b). The original upsampling circuit, prior to conversion to the quantum-forking circuit, consists of $n = 3$ qubits.

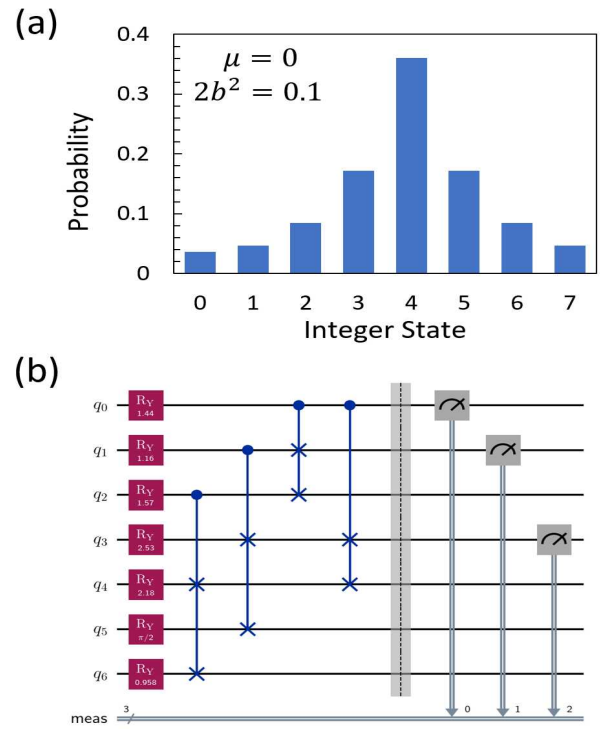

FIG. 5. (a) Laplace distributed probabilities corresponding to a state vector prepared via the quantum-forking circuit of panel (b). The original upsampling circuit, prior to conversion to the quantum-forking circuit, consists of $n = 3$ qubits.

2. *The Laplace Distribution*

The Laplace probability distribution can be expressed as

$$P(x|\mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right), \tag{44}$$

where the upsampling center of capture on the line of support is $\bar{x} = \mu$, with

$$\mu = \int_{-\infty}^{\infty} x P(x|\mu, b)\, dx, \tag{45}$$

$$b^2 = \frac{1}{2} \int_{-\infty}^{\infty} (x-\mu)^2 P(x|\mu, b)\, dx. \tag{46}$$

Figure 5(a) shows the sampling measurement of a state vector prepared using the quantum-forking circuit of Fig. 5(b). As in Fig. 4, the quantum-forking circuit was the result of converting an upsampling circuit of $n = 3$ qubits.

3. *The Lognormal Distribution*

Also, the upsampling algorithm can be applied to continuous probability distributions that can be mapped to $x \in \mathbb{R}$. An example is the lognormal probability distribution, given by

$$P(y|\mu, \sigma) = \frac{1}{y\sigma\sqrt{2\pi}} \exp\left[-\frac{(\log y - \mu)^2}{2\sigma^2}\right]. \tag{47}$$

Here, the mapping is $x = \log y$ such that $P(y|\mu, \sigma)\, dy = P(x|\mu, \sigma)\, dx$, where $P(x|\mu, \sigma)$ is the Gaussian distribution of (41). The discrete set of points sampled on the interval of $0 \le y < \infty$ are defined via $y_n = e^{x_n}$, and the width of the probability at y_n, i.e., related to the area under the curve, is $\delta y_n = e^{x_n} \delta x_n$.

B. Discrete Probability Distributions

1. *The Binomial Distribution*

As an example of a discrete case we consider the binomial distribution. Here, we define binomial probabilities in the form

$$\mathscr{P}(k, l) = \binom{l}{k} p^k q^{l-k}\,; \quad k \in \{0, 1, 2, \ldots, l\}\,, \tag{48}$$

where the support is the set of integers k, such that $0 \le k \le l$, and we define probabilities $0 \le p, q \le 1$, where $p + q = 1$.

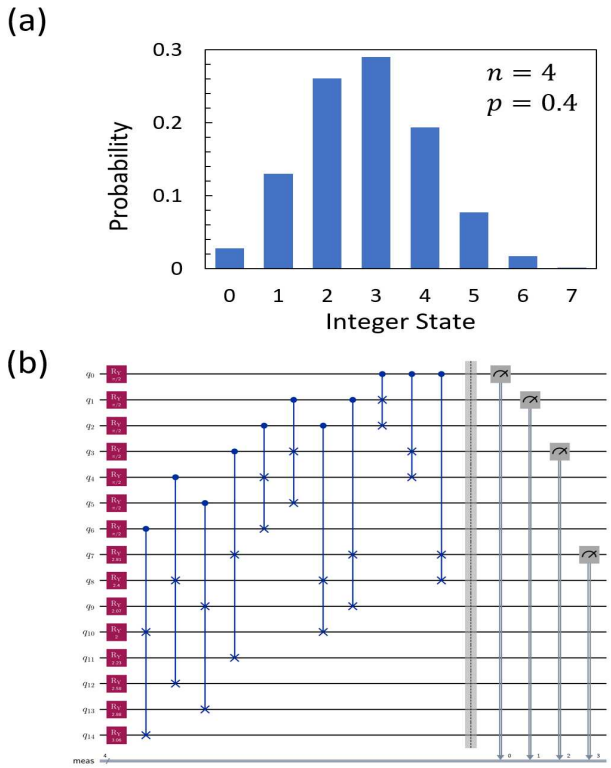

FIG. 6. (a) Plot of the binomial distribution obtained from projection of the $|0\rangle$ state of the most significant qubit of a $n = 4$ quantum circuit converted to the quantum-forking circuit depicted in panel (b).

IV. CONCLUSION

We have presented the upsampling algorithm of state-vector preparation, applicable to both continuous probability distributions with support on $x \in \mathbb{R}$, as well as arbitrary discrete probability distributions. The upsampling algorithm is a divide-and-conquer algorithm exhibiting exponential gate depth but it can be transformed to a quantum-forking algorithm of polylogarithmic gate depth [10, 11, 14]. We presented several examples, including the Gaussian, Laplace, and binomial distributions, and we showed the quantum-forking circuits we employed for our simulations.

Our analyses focused on univariate distributions, but our approach can be extended to multivariate forms. Also, the algorithm is adaptable to the preparation of state vectors associated with other functions. We hope our discussion will stimulate further interest and development.

ACKNOWLEDGMENTS

We thank V. Markov and C. Gonciulea for helpful comments and suggestions, with additional support and encouragement from E. Almond, T. Sewell, and T. Skeen. The views expressed in this article are those of the authors and do not represent the views of Wells Fargo. This article is for informational purposes only. Nothing contained in this article should be construed as investment advice. Wells Fargo makes no express or implied warranties and expressly disclaims all legal, tax, and accounting implications related to this article.

Appendix A: The Upsampling Algorithm Applied to Discrete Probabilities

We can apply the upsampling algorithm to preparation of state vectors whose coefficients are the square roots of discrete probabilities. In this case, we want to show that (34) is the PDF corresponding to state vector $|\psi_n\rangle$, as written in the form of (33). As a first step, we represent the Dirac delta function by a Gaussian whose variance approaches zero, as in (36). In our calculations that utilize (36) we omit the explicit notation of a limit, but it's understood that we must take the limit as $\epsilon \to 0$ at the end of all calculations.

Next, assuming $n > 1$, we apply (34) and (36) to (5) to obtain an expression for amplitude coefficients, $\xi_n(i_n)$, of $|\psi_n\rangle$, as expressed in the form of (4). Introducing integer k_n, of the same form as i_n, where $0 \le k_n < 2^n$, the amplitude $\xi_n(i_n)$ can be expressed as

$$\xi_n(i_n) = \left\{ \frac{\delta x_n}{2^{n-1}\sqrt{2\pi\epsilon^2}} \sum_{k_{n-1}} \left[\mathscr{P}(k_{n-1}) \sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j+(i_n-k_{n-1})/2^{n-1}\right]^2} + (1-\mathscr{P}(k_{n-1})) \sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j-1+(i_n-k_{n-1})/2^{n-1}\right]^2} \right] \right\}^{1/2} ; \quad n > 1. \tag{A1}$$

To evaluate normalization factor δx_n, we first note that $\langle\psi_n|\psi_n\rangle = 1$ implies

$$\sum_{i_n} \xi_n(i_n)^2 = 1, \tag{A2}$$

such that, inserting (A1) into this expression, then solving for δx_n, yields

$$\delta x_n = 2^{n-1}\sqrt{2\pi\epsilon^2}\Bigg\{\sum_{i_n}\sum_{k_{n-1}}\Bigg[\mathscr{P}(k_{n-1})\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j+(i_n-k_{n-1})/2^{n-1}\right]^2} + (1-\mathscr{P}(k_{n-1}))\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j-1+(i_n-k_{n-1})/2^{n-1}\right]^2}\Bigg]\Bigg\}^{-1}. \quad \text{(A3)}$$

Substituting (A3) into (A1) gives

$$\xi_n(i_n) = \Bigg(\sum_{k_{n-1}}\Bigg[\mathscr{P}(k_{n-1})\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j+(i_n-k_{n-1})/2^{n-1}\right]^2} + (1-\mathscr{P}(k_{n-1}))\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j-1+(i_n-k_{n-1})/2^{n-1}\right]^2}\Bigg] \times \Bigg\{\sum_{i_n}\sum_{k_{n-1}}\Bigg[\mathscr{P}(k_{n-1})\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j+(i_n-k_{n-1})/2^{n-1}\right]^2} + (1-\mathscr{P}(k_{n-1}))\sum_{j=-\infty}^{\infty} e^{-\left(w^2/8\epsilon^2\right)\left[2j-1+(i_n-k_{n-1})/2^{n-1}\right]^2}\Bigg]\Bigg\}^{-1}\Bigg)^{1/2}; \quad n>1. \quad \text{(A4)}$$

As $\epsilon \to 0$ on the right side of (A4), most of the terms in the sums over j vanish. The exceptional terms, which approach unity, have exponential functions with arguments that go to zero. The zero condition only occurs if the difference $i_n - k_{n-1}$ is divisible by 2^{n-1}. One can show that only $j=0$ terms are non-zero, and only when $i_n = k_{n-1}$ or $i_n = 2^{n-1}+k_{n-1}$. In particular, when $i_n = k_{n-1}$ then the most significant bit of i_n is zero, i.e., $i_{n-1}^{(n)} = 0$, such that $i_n = i_{n-1} = k_{n-1}$; when $i_n = 2^{n-1}+k_{n-1}$ then the most significant bit of i_n is unity, i.e., $i_{n-1}^{(n)} = 1$, such that $i_n = 2^{n-1}+i_{n-1}$, or $i_{n-1} = k_{n-1}$.

Taking these observations into account, then as $\epsilon \to 0$ in (A4) we obtain an intermediate result

$$\xi_n(i_n) = \Bigg(\Bigg[\mathscr{P}(i_{n-1})\,\delta_{i_{n-1}^{(n)},0} + (1-\mathscr{P}(i_{n-1}))\,\delta_{i_{n-1}^{(n)},1}\Bigg] \times \Bigg\{\sum_{i_n}\Bigg[\mathscr{P}(i_{n-1})\,\delta_{i_{n-1}^{(n)},0} + (1-\mathscr{P}(i_{n-1}))\,\delta_{i_{n-1}^{(n)},1}\Bigg]\Bigg\}^{-1}\Bigg)^{1/2}; \quad n>1. \quad \text{(A5)}$$

But, making use of the normalization condition in (27), we have

$$\sum_{i_n}\Bigg[\mathscr{P}(i_{n-1})\,\delta_{i_{n-1}^{(n)},0} + (1-\mathscr{P}(i_{n-1}))\,\delta_{i_{n-1}^{(n)},1}\Bigg] = \sum_{i_{n-1}}\Bigg[\mathscr{P}(i_{n-1}) + (1-\mathscr{P}(i_{n-1}))\Bigg] = 2^{n-1}, \quad \text{(A6)}$$

so we can reduce the expression of $\xi_n(i_n)$ to

$$\xi_n(i_n) = \sqrt{\frac{1}{2^{n-1}}\Bigg[\mathscr{P}(i_{n-1})\,\delta_{i_{n-1}^{(n)},0} + (1-\mathscr{P}(i_{n-1}))\,\delta_{i_{n-1}^{(n)},1}\Bigg]}; \quad n>1. \quad \text{(A7)}$$

Substituting (A7) into (4) yields an expression for $|\psi_n\rangle$ identical to (33). This shows that (34) is the PDF corresponding to (33).

Appendix B: The Angle Parameters of the Discrete-Probability Algorithm

The y-rotation angles, $\theta_{n,m}(i)$, of the upsampling algorithm, applicable to a state vector prepared from discrete probabilities of the form of (27), can be computed from (26) using (34). As in Appendix A, we replace the Dirac delta

functions of (34) with Gaussian functions, via (36). Expressing the finite sums as sums over integers k_{n-1}, where $0 \le k_{n-1} < 2^{n-1}$, we then have

$$\theta_{n,m}(i_{n-1}) = 2\arccos\left\{\sum_{k_{n-1}}\left[\mathscr{P}(k_{n-1})\sum_{j=-\infty}^{\infty} e^{-\left[w^2/\left(2^{2m+1}\epsilon^2\right)\right]\left[2j+(i_{n-1}-k_{n-1})/2^{n-m}\right]^2}\right.\right.$$
$$\left.\left.+\,(1-\mathscr{P}(k_{n-1}))\sum_{j=-\infty}^{\infty} e^{-\left[w^2/\left(2^{2m+1}\epsilon^2\right)\right]\left[2j-2^{m-1}+(i_{n-1}-k_{n-1})/2^{n-m}\right]^2}\right]\right\}^{1/2}$$
$$\times\left\{\sum_{k_{n-1}}\left[\mathscr{P}(k_{n-1})\sum_{j=-\infty}^{\infty} e^{-\left[w^2/\left(2^{2m+1}\epsilon^2\right)\right]\left[j+(i_{n-1}-k_{n-1})/2^{n-m}\right]^2}\right.\right.$$
$$\left.\left.+\,(1-\mathscr{P}(k_{n-1}))\sum_{j=-\infty}^{\infty} e^{-\left[w^2/\left(2^{2m+1}\epsilon^2\right)\right]\left[j-2^{m-1}+(i_{n-1}-k_{n-1})/2^{n-m}\right]^2}\right]\right\}^{-1/2}, \quad \text{(B1)}$$

where we removed explicit use of limits, with the understanding that at the end of calculation we will let $\epsilon \to 0$.

As in Appendix A, when $\epsilon \to 0$ on the right side of (B1), most of the terms in the sums over j vanish. The exceptional terms, which approach unity, have exponential functions with arguments that go to zero. The zero argument only occurs if the difference $i_{n-1} - k_{n-1}$ is divisible by 2^{n-m}, leading to subsets of values of k_{n-1} of the form $k_{n-1} = 2^{n-m} \cdot l + i_{n-1}$, corresponding to one or more integers $l \ge 0$.

When $m = 1$, the first sum over j in the numerator, proportional to $\mathscr{P}(k_{n-1})$, yields the constraint $k_{n-1} = i_{n-1}$, such that the sum over k_{n-1} reduces to the single term $\mathscr{P}(i_{n-1})$. On the other hand, the second sum over j of the numerator, proportional to $(1 - \mathscr{P}(k_{n-1}))$, is precisely zero—no terms in this sum over k_{n-1} contribute. In the denominator, when again $m = 1$, the two sums over j contribute equally, again producing the constraint $k_{n-1} = i_{n-1}$, which means we have a sum of $\mathscr{P}(i_{n-1})$ and $(1 - \mathscr{P}(i_{n-1}))$, which is unity, so the entire denominator evaluates to unity. Thus, in the $m = 1$ case, as $\epsilon \to 0$, equation (B1) simplifies to

$$\theta_{n,1}(i_{n-1}) = 2\arccos\sqrt{\mathscr{P}(i_{n-1})}. \quad \text{(B2)}$$

When $m > 1$, the two sums over j of the numerator contribute equally, and in the denominator the two sums over j also contribute equally. This means that in both numerator and denominator there are terms $\mathscr{P}(k_{n-1})$ and $(1 - \mathscr{P}(k_{n-1}))$, for values of k_{n-1} of the form $k_{n-1} = 2^{n-m} \cdot l + i_{n-1}$, that sum to unity. However, in the denominator, the count of such terms is twice that of the numerator, so in the case of $m > 1$, as $\epsilon \to 0$, equation (B1) simplifies to

$$\theta_{n,m}(i_{n-1}) = 2\arccos\left(\frac{1}{\sqrt{2}}\right) = \pi/2\,; \quad m > 1. \quad \text{(B3)}$$

Combining the two results, we can write the angle parameters of the discrete-probability algorithm as

$$\theta_{n,m}(i_{n-1}) = \begin{cases} 2\arccos\sqrt{\mathscr{P}(i_{n-1})} & ;\ m = 1 \\ \pi/2 & ;\ 1 < m \le n \end{cases}. \quad \text{(B4)}$$

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition* (Cambridge University Press, 2010) pp. 24–25.

[2] S. Aaronson, Nat. Phys. **11**, 291 (2015); J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017).

[3] L. Grover and T. Rudolph, Creating superpositions that correspond to efficiently integrable probability distributions (2002), arXiv:quant-ph/0208112 [quant-ph]; A. Carrera Vazquez and S. Woerner, Phys. Rev. Appl. **15**, 034027 (2021).

[4] C. Zoufal, A. Lucchi, and S. Woerner, npj Quantum Inf. **5**, 103 (2019); E. Y. Zhu, S. Johri, D. Bacon, M. Esencan, J. Kim, M. Muir, N. Murgai, J. Nguyen, N. Pisenti, A. Schouela, K. Sosnova, and K. Wright, Phys. Rev. Res. **4**, 043092 (2022).

[5] Y. R. Sanders, G. H. Low, A. Scherer, and D. W. Berry, Phys. Rev. Lett. **122**, 020502 (2019); J. Bausch, Quantum **6**, 773 (2022).

[6] M. Lubasch, J. Joo, P. Moinier, M. Kiffner, and D. Jaksch, Phys. Rev. A **101**, 010301 (2020); K. Plekhanov, M. Rosenkranz, M. Fiorentini, and M. Lubasch, Quantum **6**, 670 (2022); I. Y. Akhalwaya, A. Connolly, R. Guichard, S. Herbert, C. Kargi, A. Krajenbrink, M. Lubasch, C. M. Keever, J. Sorci, M. Spranger, and I. Williams, A modular engine for quantum monte carlo integration (2023), arXiv:2308.06081 [quant-ph]; J. Iaconis, S. Johri, and E. Y. Zhu, npj Quan-

tum Inf. **10**, 15 (2024); Y. Sano and I. Hamamura, Quantum state preparation for probability distributions with mirror symmetry using matrix product states (2024), arXiv:2403.16729 [quant-ph].

[7] X. Sun, G. Tian, S. Yang, P. Yuan, and S. Zhang, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **42**, 3301 (2023); X.-M. Zhang, T. Li, and X. Yuan, Phys. Rev. Lett. **129**, 230504 (2022); A. G. Rattew and B. Koczor, Preparing arbitrary continuous functions in quantum registers with logarithmic complexity (2022), arXiv:2205.00519 [quant-ph]; S. McArdle, A. Gilyén, and M. Berta, Quantum state preparation without coherent arithmetic (2022), arXiv:2210.14892 [quant-ph].

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 4th ed. (The MIT Press, 2022) pp. 76–125.

[9] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa, Quantum Info. Comput. **5**, 467–473 (2005).

[10] I. F. Araujo, D. K. Park, F. Petruccione, and A. J. da Silva, Sci. Rep. **11**, 6329 (2021).

[11] D. K. Park, I. Sinayskiy, M. Fingerhuth, F. Petruccione, and J.-K. K. Rhee, New J. of Phys. **21**, 083024 (2019).

[12] A. Kitaev and W. A. Webb, Wavefunction preparation and resampling using a quantum computer (2009), arXiv:0801.0342 [quant-ph].

[13] If ζ were varied continuously from 0 to $1/2^{n-1}$ then ergodic coverage of the entire PDF over the breadth of the sample window would be achieved.

[14] A. J. da Silva/Centro de Informática UFPE, Github—adjs/dcsp, `https://github.com/adjs/dcsp` (July 2020), accessed: November 2024.